Wake World Detection Project

The member of the group:

1. HAITHM NABIL ALI ALSAEEDI 221015221
2. Aly ABDELHALIM 211015092
3. Moatasem Morsy 211015093
4. Arshad Qurish
5. Alpha Cisse

1- Preparing Dataset:

We started with recording our voices and uploaded them to the driver in .wav extension after all students uploaded their parts, we downloaded the needed dataset to train our model.

2- Installing Libraries:

Install: pip install librosa numpy pandas scikit-learn sounddevice joblib.

Install: pip install ffmpeg-python

Libraries:

Library	Version	Purpose
librosa	≥ 0.10.0	Audio loading and feature extraction
numpy	≥ 1.23.0	Numerical operations
pandas	≥ 1.5.0	Dataset handling
scikit-learn	≥ 1.2.0	ML models, scaling, cross-validation
imbalanced-learn	≥ 0.11.0	SMOTE, BalancedRandomForest
xgboost	≥ 2.0.0	Gradient boosting in ensemble
joblib	≥ 1.2.0	Model and config serialization
matplotlib	≥ 3.6.0	Plotting (confusion matrix)
seaborn	≥ 0.12.0	Heatmap visualization
pyaudio	≥ 0.2.13	Microphone input for live detection

A. ffmpeg-python:

FFmpeg is a powerful command-line tool that handles almost every audio/video format in existence

3-Important Part to Understand:

A. STFT and MFCC:

a. **The Short-Time Fourier Transform**:

i. **Fourier Transform**: It takes a signal and tells you exactly which frequencies make it up.

ii. **Short-Time**: Speech changes over time. If I say "Wake," the "W" sound has different frequencies than the "k" sound

STFT chops the 1-second audio into tiny windows (e.g., 25 milliseconds) and calculates the frequencies for each window separately

b. **The Mel Scale**:

Humans don't hear frequencies linearly. We can easily tell the difference between 500Hz and 1000Hz, but we struggle to tell the difference between 10,000Hz and 10,500Hz. The **Mel Scale** warps the frequencies so they match how the human ear actually perceives sound

4-Creating The Dataset:

```
if label == "wake":
    noise = np.random.randn(len(audio)).astype(np.float32) * 0.005 * np.std(audio)
```

The Problem: The "Perfect" Recording

Imagine you recorded yourself saying "Wake Up" in a perfectly silent, soundproof room. If you only show the computer **that one perfect version**, it will get confused the moment you move to a kitchen with a humming fridge or a living room with a fan.

The computer becomes "spoiled"—it thinks the "Wake Up" command *only* exists in total silence.

The Solution: Adding "Noise"

To fix this, we use that line of code to **add fake background fuzz** to your recordings.

```
noise = np.random.randn(len(audio)) * 0.005 * np.std(audio)
```

Think of it like this:

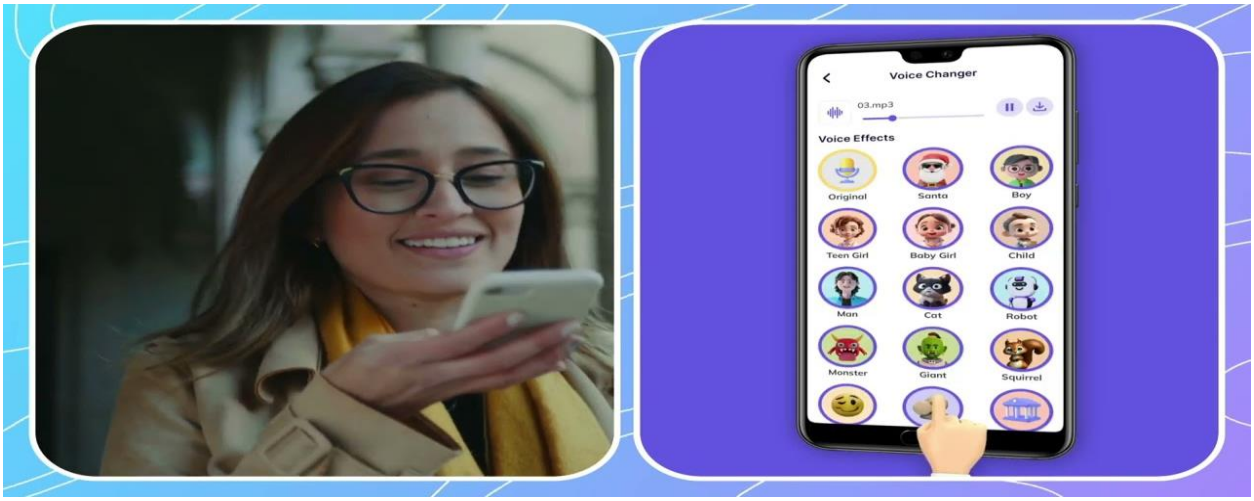
1. **`np.random.randn` (The Static):** This creates a sound that sounds like "SHHHHHHHHH." It's like the static you hear on an old TV when there's no signal.
2. **`* 0.005` (The Volume):** This makes the "SHHHHH" sound very, very quiet. If we didn't do this, the static would be a loud scream, and you wouldn't hear your voice at all. We want it to be a tiny whisper in the background.
3. **`* np.std(audio)` (The Automatic Adjuster):** This is the smartest part.
    a. If your "Wake Up" recording is **LOUD**, the code makes the static a little louder so you can still notice it.
    b. If your recording is **quiet**, the code makes the static even quieter so it doesn't swallow your voice.

"Gym for AI."

```
for aug_type, aug_audio in [
    ("original", audio.copy()),
    ("pitch_shift_up2", librosa.effects.pitch_shift(y=audio, sr=sr, n_steps=2)),
    ("pitch_shift_down2", librosa.effects.pitch_shift(y=audio, sr=sr, n_steps=-2)),
    ("pitch_shift_up1", librosa.effects.pitch_shift(y=audio, sr=sr, n_steps=1)),
    ("pitch_shift_down1", librosa.effects.pitch_shift(y=audio, sr=sr, n_steps=-1)),
    ("time_stretch_fast", librosa.effects.time_stretch(y=audio, rate=1.1)),
    ("time_stretch_slow", librosa.effects.time_stretch(y=audio, rate=0.9)),
    ("time_stretch_105", librosa.effects.time_stretch(y=audio, rate=1.05)),
    ("noise", audio + noise),
    ("reverb", apply_reverb(audio, sr)),
]:
```

Instead of just showing it one picture of a person, you are showing it that person wearing 10 different outfits so it recognizes them no matter what.

1. The Voice Changers (`pitch_shift`)



Think of these like the "helium" or "giant voice" effects on a phone app.

- **`up2` and `up1`:** These make your voice sound slightly higher-pitched.
- **`down2` and `down1`:** These make your voice sound slightly deeper.
- **Why?** You might have a cold one day (deeper voice), or a child might try to use the system (higher voice). This teaches the computer that "Wake Up" is still the same command even if the **pitch** changes

How the Computer Does It

The computer has to change the **pitch** (how high or low you sound) without changing the **speed** (how fast you talk).

If you just sped up a vinyl record, the voice would get higher but also much faster (like a chipmunk). To prevent this, `librosa` uses a "Slice and Stretch" method:

1. **Slicing:** It cuts your audio into thousands of tiny, microscopic grains.
2. **Shifting:** It mathematically "squeezes" the waves inside those grains to make them vibrate faster (higher pitch).
3. **Smoothing:** It glues them back together so you don't hear any clicks or gaps.

2. The Speed Controls (`time_stretch`)

Think of this like a YouTube video playing at **0.9x** or **1.1x** speed.

- **`fast` (1.1) and `105` (1.05):** These speed up the recording slightly.
- **`slow` (0.9):** This slows down the recording.
- **Why?** Sometimes you are in a rush and say "WakeUp!" really fast. Other times you might be tired and say "Waaake Uuup" slowly. This teaches the computer to recognize the **pattern**, regardless of the **speed**.

3. The "Copy" (`original`)

- **What it is:** This is just your original, clean recording.
- **Why?** We still need the computer to know what you *actually* sound like in perfect conditions!

4. The "Room" Effect (`reverb`)

Think of this like the difference between talking into a pillow versus talking in a **large, empty bathroom**.

- **What it does:** It adds a "ringing" or "echo" to the sound.
- **Why?** In a real house, sound bounces off walls and tiles. This "reverb" teaches the computer to hear through those echoes so it doesn't get confused by the acoustics of a room.

Putting it all together:

For every **1** file you recorded, this loop creates **10** different versions:

1. Original
2. High Voice (+2)
3. Slightly High Voice (+1)
4. Deep Voice (-2)
5. Slightly Deep Voice (-1)

6. Fast Talking (1.1)
7. Medium-Fast Talking (1.05)
8. Slow Talking (0.9)
9. TV Static Background (`noise`)
10. Bathroom Echo (`reverb`)

Hard Negatives

A **Hard Negative** is a sound that *sounds* like a wake word but isn't.

- **Example:** You say "Hi Pakize" and the AI accidentally thinks you said "Hey Pakize"
- You save these "mistake" sounds into a special folder called `hard_negatives`

The "Study Harder" Loop

For every 1 mistake sound you find, the code creates **15 different versions** of it.

- **Why?** We want the AI to be *very* sure that this sound is wrong. By making 15 versions, we are flooding the "Non-Wake" category with examples of what a "Wake" word **does not** sound like.

Dataset Creation:

```
delta2_cols = [f"delta2_{i}" for i in range(13)] + [f"delta2_std_{i}" for i in range(13)]
contrast_cols = [f"contrast_{i}" for i in range(7)] + [f"contrast_std_{i}" for i in range(7)]
mel_cols = [f"mel_mean_{i}" for i in range(40)] + [f"mel_std_{i}" for i in range(40)]
chroma_cols = [f"chroma_mean_{i}" for i in range(12)]
FEATURE_COLUMNS = (
    [f"mfcc_{i}" for i in range(13)] + [f"mfcc_std_{i}" for i in range(13)]
    + [f"delta_{i}" for i in range(13)] + delta2_cols + contrast_cols
    + ["zcr_mean", "zcr_std", "rms_mean"] + mel_cols + chroma_cols
)
columns = FEATURE_COLUMNS + ["label", "file_path"]
df = pd.DataFrame(data, columns=columns)

df.to_csv("dataset.csv", index=False)
pd.DataFrame(manifest, columns=["path", "label", "aug_type"]).to_csv("dataset_manifest.csv", index=False)

print("dataset.csv created")
print(f"Class counts: {df['label'].value_counts().to_dict()}")
```

mel_std_3	mel_std_3	mel_std_3	mel_std_3	mel_std_3	mel_std_3	mel_std_3	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	chroma_m	label
12.01421	12.5673	12.78283	12.34042	12.24343	12.16853	11.61272	0.417144	0.369614	0.463154	0.457401	0.337219	0.339345	0.366716	0.463128	0.538545	0.390988	0.335789	0.443443	wake
10.38093	10.25946	10.91555	11.86784	11.91439	11.62103	10.72733	0.275389	0.445776	0.359735	0.269264	0.378901	0.478383	0.309178	0.28026	0.30537	0.461406	0.570936	0.302284	wake
11.63441	11.46141	11.29696	10.7275	10.55649	9.929703	2.356622	0.417932	0.419147	0.330894	0.354203	0.39711	0.58628	0.534045	0.361418	0.427263	0.52282	0.359625	0.322086	wake
10.38568	11.002	12.05495	12.20137	11.69302	11.26862	10.73279	0.416437	0.241029	0.354981	0.479869	0.356684	0.288494	0.316956	0.416846	0.63883	0.46887	0.282527	0.347232	wake
11.30912	11.86744	11.46534	11.10658	10.9959	10.85073	8.019609	0.365559	0.460595	0.497866	0.309131	0.336374	0.374713	0.41891	0.566549	0.371053	0.323397	0.460426	0.452957	wake
10.17987	10.8133	11.36005	10.87312	10.51862	10.39821	9.950085	0.337724	0.351153	0.446815	0.372121	0.353385	0.351401	0.405526	0.574369	0.468851	0.303963	0.449415	0.512045	wake
10.51744	10.60065	11.1548	10.63686	10.55136	10.4073	9.585526	0.371333	0.273245	0.363068	0.48187	0.30971	0.320582	0.327865	0.422871	0.557402	0.362717	0.253494	0.307715	wake
11.32785	11.76415	12.08331	11.67894	11.42781	11.3175	10.99895	0.408441	0.332462	0.408268	0.479265	0.306411	0.316048	0.353266	0.447151	0.569825	0.380819	0.312162	0.435701	wake
8.912553	9.377584	9.771561	9.447994	9.050012	8.961934	8.389544	0.416017	0.372664	0.46956	0.461985	0.342149	0.343303	0.365686	0.460296	0.540134	0.390045	0.340021	0.445368	wake
11.47267	11.60803	11.78114	11.38984	11.28541	11.22402	10.3608	0.405111	0.221641	0.302855	0.420561	0.319389	0.312524	0.321323	0.386773	0.607185	0.374314	0.238367	0.326256	wake
15.74231	12.6824	11.17839	11.03706	11.31678	12.63465	14.33836	0.652894	0.488792	0.55673	0.618202	0.624673	0.694921	0.717893	0.703217	0.640661	0.651033	0.749394	0.863235	wake
19.7934	17.58069	15.74426	12.14942	11.22057	11.9931	10.54191	0.639389	0.781253	0.511638	0.432553	0.476309	0.557429	0.613672	0.681665	0.699824	0.695417	0.617825	0.529159	wake

Summary of the Process

Step	Goal	Key Line
Catchi ng	Find where the AI "lied."	`df[(df['label'] == 'nonwake') & (df['probability'] >= 0.7)]`
Labeling	Tell the AI it was a mistake.	`row = extract_features(aug, sr) + ["nonwake", ...]`
Scaling	Make the mistake impossible to ignore.	`for i in range(15):`

Extracting Feature:

```
if len(audio) < max_len:
    audio = np.pad(audio, (0, max_len - len(audio)))
else:
    audio = audio[:max_len]
```

- **The Goal:** Computers need every input to be exactly the same size.
- **The Logic:** If the sound "Hey Pakize" is too short, we add silence at the end (**pad**). If it's too long, we snip it off (**audio[:max_len]**). This ensures every "test" the model takes is exactly 1 second long

```
mfcc = librosa.feature.mfcc(y=audio, sr=sample_rate, n_mfcc=13)
delta = librosa.feature.delta(mfcc)
delta2 = librosa.feature.delta(mfcc, order=2)
```

- **MFCCs:** These are like the **"Fingerprints"** of a voice. They describe the shape of your vocal tract while you talk.
- **Deltas:** These measure **how the sound is changing**. `delta` is the first change (speed), and `delta2` is the second change (acceleration). It helps the model understand the transition from "Hey" to "Pakize."

contrast = librosa.feature.spectral_contrast(y=audio, sr=sample_rate, n_bands=6)

- **The Goal:** This measures the difference between the "peaks" (loudest parts) and "valleys" (quietest parts) of the sound frequencies.

- **Simple Example:** It helps the model tell the difference between a "clear" voice and a "fuzzy" background noise.

```
zcr = librosa.feature.zero_crossing_rate(audio)
zcr_mean = float(np.mean(zcr))
zcr_std = float(np.std(zcr))
```

- **ZCR (Zero Crossing Rate):** This counts how many times the sound wave crosses the center line. High ZCR sounds like "shhh" or "ssss" (hissing). Low ZCR sounds like a deep hum.
- **RMS (Root Mean Square):** This is simply the **Volume**. It tells the model how much energy/power is in the sound.

5- Training Model:

```
try:
    from imblearn.ensemble import BalancedRandomForestClassifier
    from imblearn.over_sampling import SMOTE
    USE_BALANCED_RF = True
    USE_SMOTE = True
except ImportError:
    USE_BALANCED_RF = False
    USE_SMOTE = False

try:
    from xgboost import XGBClassifier
    from sklearn.base import BaseEstimator, ClassifierMixin
    USE_XGB = True
except ImportError:
    USE_XGB = False
```

- **The Problem:** We have way more "nonwake" (noise) sounds than "wake" sounds. If a student takes a test where 90% of the answers are "B," they will just learn to circle "B" every time without reading the questions.
-
- **SMOTE (The Leveler):** This is a tool that **creates fake, smart copies** of "wake" recordings so the "wake" and "nonwake" groups are the same size.
-
- **BalancedRandomForest:** This is a special type of AI "brain" that is specifically designed to not ignore the smaller group (the "wake" words).
-
  XGBoost: In the world of AI, this is like the "Valedictorian." It is a very powerful and fast model that is famous for winning coding competitions.
-
- **Why use it?** It's great at finding the tiny, tiny differences between "Hey Pakize" and "Hey Istanbul."

Tool	Role	Simple Purpose
SMOTE	The Multiplier	Makes "fake" copies of your voice so the data is balanced.
Balanced RF	The Fair Group	100 mini-judges who refuse to ignore the smaller group.
XGBoost	The Expert	Focuses purely on fixing mistakes to get the highest score.
Blueprints	The Template	The "standard rules" so all models work the same way.

SVM theory, the model wants to minimize "Loss." Because we have more nonwake data, the model found that the "easiest" way to reduce loss was to draw the decision boundary so far away that everything falls into the nonwake zone.

```
df = pd.read_csv("dataset.csv")
X = df.drop(columns=["label", "file_path"], errors="ignore").select_dtypes(include=[np.number])
y = df["label"]
drop_cols = [c for c in X.columns if c in ("mfcc_0", "mfcc_std_0", "delta_0", "delta2_0", "delta2_std_0")]
X = X.drop(columns=drop_cols, errors="ignore")
```

- **(The Clues):** These are the thousands of numbers (features) we extracted from the audio. We "drop" (remove) the `label` and `file_path` because we don't want the AI to see the answer while it's looking at the clues.

- **y (The Answer Key):** This is just a list of the labels ("wake" or "nonwake"). This is the "Truth" we use to grade the AI later.
- **`select_dtypes`:** This is a safety filter. It says, "Only give me the columns that are actual numbers." If there is a random piece of text in there, the AI can't do math with it, so we ignore it.

- **The Problem:** In audio processing, the very first number (the "0" index) for things like MFCCs usually just represents the **overall volume** or "energy" of the file.
-
- **Why it's a problem:** If you train the AI using volume, it will think that *any* loud sound is "Hey Pakize" and *any* quiet sound is "nonwake."
- **The Solution:** We explicitly delete these columns (`mfcc_0`, etc.). This forces the AI to look at the **texture and pattern** of your voice (the actual name "Pakize") instead of just how loud the recording is.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

- The scaler "studies" the training data to learn what is "normal" for voice (e.g., "The average pitch is 200").

- **`transform`:** It then subtracts that average and divides by the spread.

- **The Result:** All your numbers now live in a small range (usually between -3 and +3). Now the AI can compare "Volume" to "Pitch" without getting confused by the size of the numbers.

```python
if USE_SMOTE:
    train_counts = pd.Series(y_train).value_counts()
    k = min(5, int(train_counts.min()) - 1)
    if k < 1:
        X_train_bal, y_train_bal = X_train_scaled, y_train
    else:
        smote = SMOTE(sampling_strategy="minority", random_state=42, k_neighbors=k)
        X_train_bal, y_train_bal = smote.fit_resample(X_train_scaled, y_train)
    print(f"SMOTE: {len(y_train)} -> {len(y_train_bal)} samples (wake balanced)")
else:
    X_train_bal, y_train_bal = X_train_scaled, y_train
```

Train_count contains the amount for wake and nonwake counts, for example, 200 nonwake and 10 wke

To create a "fake" copy of your voice, SMOTE needs to look at your existing "Hey Pakize" recordings to see how they are similar. Basically, it will look for 5 wake records and study them to see how similar they are, so it can create fake data like them.

`sampling_strategy="minority"`: giving a specific instruction: "Find the group with fewer people (the **1,251** 'Wake' samples) and make more of them until they match the big group (Nonwake' samples)."

```python
n_select = min(N_FEATURES_RFE, X_train_bal.shape[1])
if n_select >= X_train_bal.shape[1]:
    selected_mask = np.ones(X_train_bal.shape[1], dtype=bool)
    selected_cols = X_train.columns.tolist()
    X_train_sel, X_val_sel, X_test_sel = X_train_bal, X_val_scaled, X_test_scaled
else:
    rfe_rf = RandomForestClassifier(n_estimators=100, random_state=42, class_weight="balanced")
    rfe = RFE(rfe_rf, n_features_to_select=n_select, step=1, verbose=0)
    rfe.fit(X_train_bal, y_train_bal)
    selected_mask = rfe.support_
    selected_cols = X_train.columns[selected_mask].tolist()
    X_train_sel = X_train_bal[:, selected_mask]
    X_val_sel = X_val_scaled[:, selected_mask]
    X_test_sel = X_test_scaled[:, selected_mask]
print(f"RFE: selected top {n_select} features")
```

The Logic: You want 65 features. But if your data somehow only has 50 features total, you can't pick 65. This line ensures you don't try to pick more than you actually have.

```
rfe_rf = RandomForestClassifier(n_estimators=100, random_state=42,
class_weight="balanced")
```

- It’s like we hire a **Random Forest** to be the judge. Why? Random Forests are great at telling you which clues are actually helping them solve a puzzle and which clues are useless. The judge looks at all 150 features. It finds the **single worst feature** (the one that helps the least) and kicks it out (`step=1`). It repeats this over and over until only the **top 65** are left.

But why 65? Well, there isn't a "magic" reason why 65 is better than 70 without testing it.

The "Goldilocks" Zone (Overfitting vs. Underfitting)

We are looking for the "Just Right" amount of information:

- **Too Few (e.g., 10 features):** The model is "too simple" and misses the details of your voice. (Underfitting)
- **Too Many (e.g., 150 features):** The model starts memorizing the background static or the hum of your laptop instead of your voice. (Overfitting)

- **65 vs 70:** Usually, we test several numbers. If the accuracy at 65 features is **85.7%** and the accuracy at 70 features is also **85.7%**, we choose **65**. Why? Because a simpler model is faster and less likely to break when it hears a new noise.

Hyperparameter Tuning.

```
svc_grid = GridSearchCV(
    SVC(kernel="rbf", probability=True, class_weight="balanced"),
    {"C": [0.1, 1, 10, 100], "gamma": [1e-3, 1e-4, "scale"]},
    cv=5, scoring="recall_macro", n_jobs=-1, verbose=1,
)
svc_grid.fit(X_train_sel, y_train_bal)

RFCls = BalancedRandomForestClassifier if USE_BALANCED_RF else RandomForestClassifier
rf_kwargs = {"random_state": 42} if USE_BALANCED_RF else {"class_weight": "balanced", "random_state": 42}
rf_grid = GridSearchCV(
    RFCls(**rf_kwargs),
    {"n_estimators": [150, 250], "max_depth": [12, 16, 20], "min_samples_leaf": [2, 3]},
    cv=5, scoring="recall_macro", n_jobs=-1, verbose=1,
)
rf_grid.fit(X_train_sel, y_train_bal)
```

The **SVC** and the **Random Forest**) to try hundreds of different study strategies to see which one gets the highest score.

- **SVC (Support Vector Classifier):** This is a **"Boundary Drawer."** It tries to draw a clear line in the math to separate "Hey Pakize" from "Noise."

- **The "C" and "Gamma":** These are the **knobs and dials**.

- **C** controls how much we punish the AI for making mistakes.

- **Gamma** controls how "curvy" the boundary line should be.

- **GridSearchCV:** This is the **"Automatic Tester."** It will try every combination (C=0.1 with Gamma=1e-3, C=1 with Gamma=1e-3, etc.) to find the winner.

- **Random Forest:** This is a **"Committee of Judges."** It uses hundreds of mini-decision trees to vote on the sound.
-
- **The Settings:**
    - **n_estimators:** Should we have 150 judges or 250 judges?

    - **max_depth:** How "deep" or complex should each judge's logic be? (12 levels of "If/Else" or 20?)

    - **min_samples_leaf:** How many examples does a judge need to see before they are allowed to make a rule?
    - **cv=5 (Cross-Validation):** The computer splits the data into 5 different piles. It trains on 4 and tests on 1. It repeats this 5 times. This ensures the model didn't just get a "lucky" batch of data.

```python
if USE_XGB:
    scale_pos = (y_train_bal == "nonwake").sum() / max((y_train_bal == "wake").sum(), 1)
    xgb_grid = GridSearchCV(
        XGBWrapper(scale_pos_weight=scale_pos),
        {"n_estimators": [200, 400], "max_depth": [4, 6], "learning_rate": [0.05, 0.1]},
        cv=5, scoring="recall_macro", n_jobs=-1, verbose=1,
    )
    xgb_grid.fit(X_train_sel, y_train_bal)
    estimators_for_stack = [
        ("svc", svc_grid.best_estimator_),
        ("rf", rf_grid.best_estimator_),
        ("xgb", xgb_grid.best_estimator_),
    ]
    print("Using XGBoost in ensemble")
else:
    gbc_grid = GridSearchCV(
        GradientBoostingClassifier(random_state=42),
        {"n_estimators": [200, 400], "max_depth": [5, 7], "learning_rate": [0.03, 0.05]},
        cv=5, scoring="recall_macro", n_jobs=-1, verbose=1,
    )
    gbc_grid.fit(X_train_sel, y_train_bal)
    estimators_for_stack = [
        ("svc", svc_grid.best_estimator_),
        ("rf", rf_grid.best_estimator_),
        ("gbc", gbc_grid.best_estimator_),
    ]
```

First, we check if the SMOTE balanced the samples. This is a layer of protection; if SMOTE balanced the samples, then the scale_pos = 1. But if the scale_pos !=1, like 1.75, that tells the machine if you miss a nonwake then –1 point, but if you miss a wake then –1.75 point, like a punishment.

```python
estimators_for_stack = [
    ("svc", svc_grid.best_estimator_),
    ("rf", rf_grid.best_estimator_),
    ("xgb", xgb_grid.best_estimator_),
]
```

Instead of picking the "one best model" and throwing the others away, `estimators_for_stack` puts all three winners into a list. This is like a **Supreme Court** where three judges will listen to your voice and vote.

```python
model = StackingClassifier(
    estimators=estimators_for_stack,
    final_estimator=LogisticRegression(random_state=42, max_iter=1000, class_weight="balanced"),
    cv=5, n_jobs=-1,
)
model.fit(X_train_sel, y_train_bal)
```

The Final Estimator: Instead of just averaging them, you hire a **Final Judge** (Logistic Regression). This judge learns whose opinion to trust more.

Example: If the SVC is usually right about noisy rooms but the XGB is better at quiet ones, the Final Judge learns to listen to the SVC when it sees high background noise features.

```python
for thr in np.arange(0.15, 0.71, 0.01):
    pp = np.where(proba_val > thr, "wake", "nonwake")
    tp_ = int(((y_val == "wake") & (pp == "wake")).sum())
    fn_ = int(((y_val == "wake") & (pp == "nonwake")).sum())
    fp_ = int(((y_val == "nonwake") & (pp == "wake")).sum())
    wr = tp_ / max(tp_ + fn_, 1)
    if fp_ < fallback_fp:
        fallback_thr, fallback_fp, fallback_wr = float(thr), fp_, wr
    if wr >= 0.75 and fp_ < best_fp:
        best_fp, best_thr, best_wr = fp_, float(thr), wr
    elif wr >= 0.75 and fp_ == best_fp and wr > best_wr:
        best_thr, best_wr = float(thr), wr
```

- **The Loop:** The code tests every level from 0.15 to 0.71.
- **The Goal:** It looks for the **"Sweet Spot"** where:
- **Wake Recall (wr) >= 75%**: It catches at least 75% of your "Hey Pakize" calls.
- **Minimum FP (False Positives)**: It makes the fewest possible "False Alarms" (waking up when you didn't call it).

The result of the first try:

```
--- Test set (threshold-optimized) ---
Accuracy: 0.8568
              precision    recall  f1-score   support

     nonwake       0.91      0.88      0.90       541
        wake       0.73      0.79      0.76       220

    accuracy                           0.86       761
   macro avg       0.82      0.84      0.83       761
weighted avg       0.86      0.86      0.86       761

Saved confusion_matrix.png
Hard negative mining: 63 FP -> 35 files -> hard_negatives.txt

>>> Wake recall: 79.1%  Accuracy: 0.857  Threshold: 0.240 <<<
```

Explaining:

- **Precision** answers: "Of all the times the model predicted Wake
  - **High Precision:** When the model triggers, it's almost certainly the wake word.
  - **Low Precision:** The model triggers for random noises, coughing, or other words.
  - **Image Result (0.00):** This means the model never successfully identified a single "Wake" word correctly.

- **Recall** answers: "Of all the actual 'Wake' sounds in the dataset, how many did the model find?
  - **High Recall:** The model catches every "Hey Pakize" you say.
  - **Low Recall:** You have to scream the wake word 10 times before it hears you once.
  - **Image Result (0.00):** This is the biggest red flag. The model "missed" every single positive case in your test set.

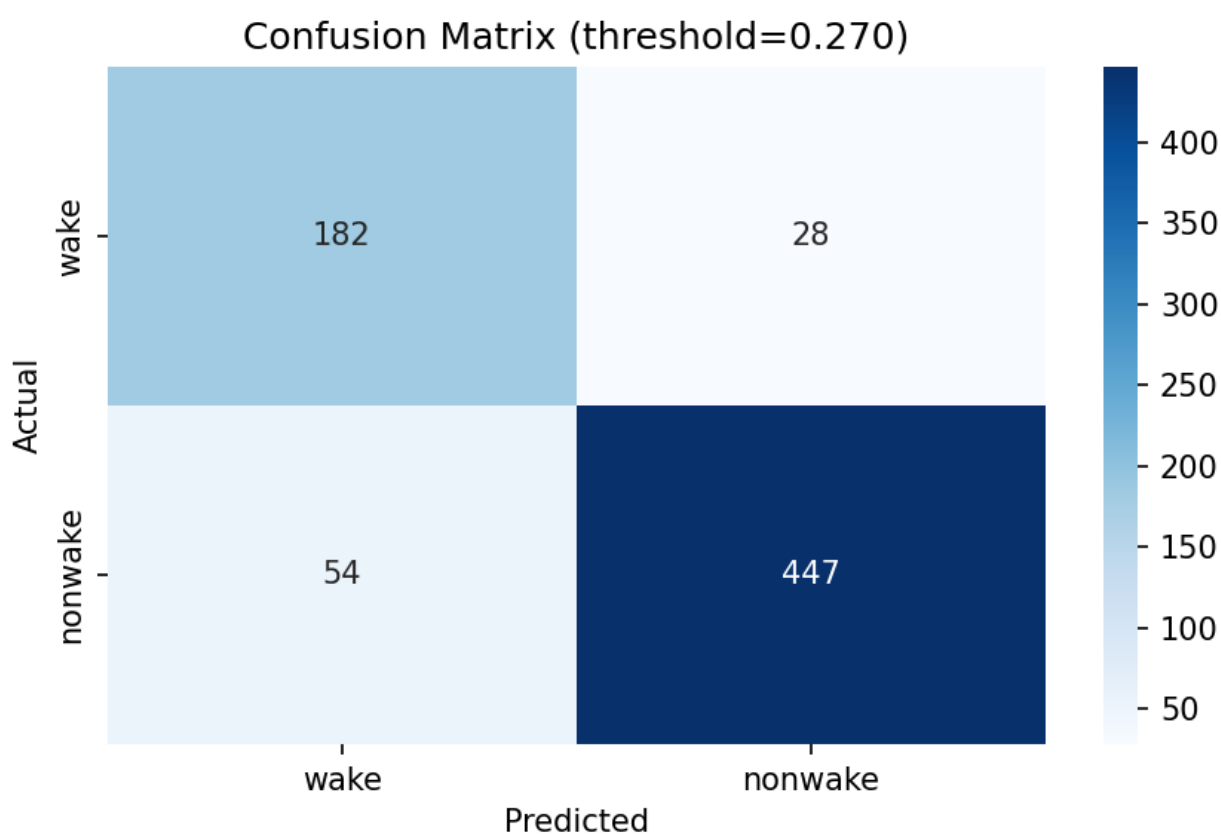
- **F1-Score** is the harmonic mean of Precision and Recall

- It is a single number that punishes extreme values. If your Precision is 1.0 but your Recall is 0.0, your F1-score will be 0

- **Support** is simply the **count** of samples for that class in your testing set
- **Accuracy** is the total number of correct predictions divided by the total number of samples. Accuracy = Correct Prediction / Total Samples

- **Macro Average** calculates the metric for each class and then takes the simple average.
  - **Formula:** ({F1 of Wake} + {F1 of Nonwake}) / 2
  - **Why it matters:** It treats "Wake" and "Nonwake" as equally important, regardless of how many samples you have of each
  - Image Macro Avg F1 is 36 which tells that the Model failed half way

Confusion Matrix:



Wake Word Detection

Real-time wake word detection ("Hey Pakize") using a stacking ensemble trained on acoustic features. Designed for low-latency, offline-capable deployment.

Overview

This project implements a keyword spotting (KWS) system that:

- Detects a custom wake phrase in real time from microphone input
- Uses traditional ML (no deep learning) for fast inference and small footprint
- Supports an iterative workflow to reduce false positives via hard negative mining
- Produces evaluation reports (ROC/PR curves, FP/hour) for tuning

Architecture

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│  Microphone  │────▶│ Sliding 1s   │────▶│ Feature      │
│  (16 kHz)    │     │ window 0.25s │     │ Extraction   │
└──────────────┘     └──────────────┘     └──────────────┘
                                                  │
                                                  ▼
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│   Trigger    │◀────│ N consecutive│◀────│ Stacking     │
│   (output)   │     │ windows > θ  │     │ Ensemble     │
└──────────────┘     └──────────────┘     └──────────────┘
```

Components

Component	Description
Features	MFCC (13), deltas, spectral contrast (7), mel (40), chroma (12), ZCR, RMS
Model	Stacking: SVC + Random Forest + XGBoost → Logistic Regression

Inference	Threshold tuning, optional VAD gate, cooldown after trigger
Config	Single `config.yaml` for paths, audio, training, realtime params

Pipeline

1. **Dataset** – Build feature CSV from `dataset/wake/` and `dataset/nonwake/` with augmentation
2. **Train** – SMOTE, RFE (65 features), GridSearchCV, threshold optimization
3. **Collect** – Copy false positives to `hard_negatives/` for next iteration
4. **Evaluate** – ROC/PR curves, metrics, FP/hour estimate
5. **Realtime** – Live mic with VAD, cooldown, sequential window requirement

Requirements

- **Python** 3.8+
- See `requirements.txt` for pinned versions

Installation

```
pip install -r requirements.txt
```

Optional: Install in editable mode for CLI:

```
pip install -e .
```

Windows – pyaudio: If `pip install pyaudio` fails:

```
pip install pipwin
pipwin install pyaudio
```

FFmpeg (optional): For MP3/non-WAV support. WAV-only needs `soundfile` only.

Quick Start

```
# 1. Prepare data
#    Add .wav files to dataset/wake/ and dataset/nonwake/

# 2. Build dataset and train
python run_wakeword.py dataset
python run_wakeword.py train

# 3. Run live detection
python run_wakeword.py realtime
```

Usage

CLI Commands

Command	Description
`dataset`	Build dataset.csv from audio folders
`train`	Train model, save artifacts, run evaluation
`evaluate`	Generate ROC/PR report (standalone)
`realtime`	Live microphone detection
`file-test`	Test .wav files in test_samples/
`collect`	Copy hard_negatives.txt entries to hard_negatives/

```
python run_wakeword.py <command>
# or after pip install -e .:
wakeword <command>
```

Main Commands

Task	Command
Live mic detection	`python run_wakeword.py realtime -t 0.70 -s 2`
Test .wav files	`python run_wakeword.py file-test -t 0.70 -s 2`

Rebuild dataset	`python run_wakeword.py dataset`
Train model	`python run_wakeword.py train`
Evaluation report	`python run_wakeword.py evaluate`
Mine hard negatives	`python run_wakeword.py mine -t 0.5`

Configuration

Edit `config.yaml` to adjust:

- **paths** – dataset, model, evaluation output
- **audio** – sample_rate, max_len, n_mfcc, n_mels
- **dataset** – augmentation list, hard_neg count
- **train** – RFE features, threshold search, min recall
- **realtime** – VAD, cooldown, sequential windows

Evaluation Report

After training (or via `wakeword evaluate`), output appears in `evaluation_report/`:

- `report.md` – Metrics summary
- `report.json` – Machine-readable metrics
- `roc_pr_curves.png` – ROC and PR curves
- `confusion_matrix_eval.png` – Confusion matrix

Model Comparison

Baseline	Typical performance	Notes
Current (Stacking)	~79% wake recall, ~86% accuracy	SVC+RF+XGB, RFE, threshold tuning
Single SVC	Lower recall	Simpler, faster
Single RF	Similar	No probability calibration

XGBoost only	Comparable	Single model, faster train

For production, consider:

- **Smaller feature set** – Fewer RFE features for lower latency
- **Quantization** – Export to ONNX/OpenVino for edge
- **Deep model** – Small CNN/RNN for higher accuracy (larger footprint)

Testing

```
pytest tests/ -v
```

Tests cover: feature extraction, inference path, threshold/cooldown logic, VAD.

Project Structure

```
wake_world_project/
├── config.yaml
├── src/wakeword/
│   ├── config.py, features.py, dataset.py
│   ├── train.py, inference.py, evaluate.py
│   ├── vad.py, realtime.py, file_test.py, collect.py
│   └── cli.py
├── tests/
├── run_wakeword.py
├── create_dataset.py, train_model.py, ...  # legacy wrappers
├── evaluation_report/
├── dataset/, hard_negatives/, test_samples/
└── model.pkl, scaler.pkl, inference_config.pkl
```

Future Work

End-to-end deep model (e.g., small CNN on mel spectrogram)

Multi-wake-word support

Export to ONNX / TensorFlow Lite for mobile

Speaker adaptation / personalization

Quantized inference for MCU deployment